

Skill 4:

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_TOKENS 50
#define MAX_LEN 100

typedef struct {
    char value[MAX_LEN];
    char type[20];
} Token;

Token tokens[MAX_TOKENS];
int token_count = 0;

/* ----- LEXICAL ANALYZER ----- */

void add_token(char *value, char *type) {
    if (token_count >= MAX_TOKENS) {
        printf("Error: Too many tokens.\n");
        return;
    }

    strcpy(tokens[token_count].value, value);
    strcpy(tokens[token_count].type, type);
    token_count++;
}

void tokenize(char *input) {
    int i = 0;
    char buffer[MAX_LEN];

    while (input[i] != '\0') {

        /* Ignore whitespace */
        if (isspace(input[i])) {

```

```

        input[i] == '|' ||
        input[i] == '&') {

        buffer[0] = input[i];
        buffer[1] = '\0';

        if (input[i] == ';')
            add_token(buffer, "DELIMITER");
        else if (input[i] == '|')
            add_token(buffer, "PIPE");
        else
            add_token(buffer, "BACKGROUND");

        i++;
        continue;
    }

    /* Normal word/token */
    int j = 0;

    while (input[i] != '\0' &&
           !isspace(input[i]) &&
           input[i] != ';' &&
           input[i] != '|' &&
           input[i] != '&') {

        buffer[j++] = input[i++];
    }

    buffer[j] = '\0';

    add_token(buffer, "WORD");
}

/* ----- TOKEN VALIDATION ----- */

```

```

/* ----- PARSER ----- */
void parse_command() {
    printf("\n--- PARSE TREE ---\n");
    printf("COMMAND\n");
    for (int i = 0; i < token_count; i++) {
        if (strcmp(tokens[i].type, "WORD") == 0) {
            printf("    └─ WORD: %s\n", tokens[i].value);
        } else if (strcmp(tokens[i].type, "PIPE") == 0) {
            printf("    └─ PIPE: |\n");
        } else if (strcmp(tokens[i].type, "DELIMITER") == 0) {
            printf("    └─ DELIMITER: ;\n");
        } else if (strcmp(tokens[i].type, "BACKGROUND") == 0) {
            printf("    └─ BACKGROUND: &\n");
        }
    }
    printf("\nSyntax validation successful.\n");
}

/* ----- MAIN ----- */
int main() {
    |   char input[500];

```

```

/* ----- MAIN ----- */

int main() {

    char input[500];

    printf("Enter command: ");
    fgets(input, sizeof(input), stdin);

    /* Remove newline */
    input[strcspn(input, "\n")] = '\0';

    printf("\nInput: %s\n", input);

    /* Tokenization */
    tokenize(input);

    /* Display tokens */
    display_tokens();

    /* Validate */
    if (!validate_tokens()) {
        printf("\nParsing stopped due to syntax error.\n");
        return 1;
    }

    /* Parse */
    parse_command();

    return 0;
}

```

```
bhanu@DESKTOP-DE32HQ8:~$ cd ~/fifo_project/fifo_project
bhanu@DESKTOP-DE32HQ8:~/fifo_project/fifo_project$ pwd
/home/bhanu/fifo_project/fifo_project
bhanu@DESKTOP-DE32HQ8:~/fifo_project/fifo_project$ nano parser.c
bhanu@DESKTOP-DE32HQ8:~/fifo_project/fifo_project$
bhanu@DESKTOP-DE32HQ8:~/fifo_project/fifo_project$
bhanu@DESKTOP-DE32HQ8:~/fifo_project/fifo_project$ gcc parser.c -o parser
bhanu@DESKTOP-DE32HQ8:~/fifo_project/fifo_project$ ./parser
Enter command: ls -l /home
```

Input: ls -l /home

--- TOKEN STREAM ---

Token 1: ls	Type: WORD
Token 2: -l	Type: WORD
Token 3: /home	Type: WORD

--- PARSE TREE ---

COMMAND

- └─ WORD: ls
- └─ WORD: -l
- └─ WORD: /home

Syntax validation successful.

```
bhanu@DESKTOP-DE32HQ8:~/fifo_project/fifo_project$ ./parser
Enter command: ls;pwd
```

Input: ls;pwd

--- TOKEN STREAM ---

Token 1: ls	Type: WORD
Token 2: ;	Type: DELIMITER
Token 3: pwd	Type: WORD

--- PARSE TREE ---

COMMAND

- └─ WORD: ls
- └─ DELIMITER: ;
- └─ WORD: pwd

Syntax validation successful.

```
Syntax validation successful.
bhanu@DESKTOP-DE32HQ8:~/fifo_project/fifo_project$ ./parser
Enter command: | ls

Input: | ls

--- TOKEN STREAM ---
Token 1: |                Type: PIPE
Token 2: ls               Type: WORD
Syntax Error: Pipe cannot be at the beginning or end.

Parsing stopped due to syntax error.
bhanu@DESKTOP-DE32HQ8:~/fifo_project/fifo_project$ ./parser
Enter command: ls || grep

Input: ls || grep

--- TOKEN STREAM ---
Token 1: ls               Type: WORD
Token 2: |                Type: PIPE
Token 3: |                Type: PIPE
Token 4: grep             Type: WORD
Syntax Error: Consecutive pipes are not allowed.

Parsing stopped due to syntax error.
bhanu@DESKTOP-DE32HQ8:~/fifo_project/fifo_project$ ls || grep
client client.c parser parser.c server server.c
bhanu@DESKTOP-DE32HQ8:~/fifo_project/fifo_project$ ./parser
Enter command: ./parser

Input: ./parser

--- TOKEN STREAM ---
Token 1: ./parser         Type: WORD

--- PARSE TREE ---
COMMAND
└─ WORD: ./parser

Syntax validation successful.
bhanu@DESKTOP-DE32HQ8:~/fifo_project/fifo_project$ |
```